ARTIFICIAL INTELLIGENCE

TOPIC 4.1

CNN supplement: residual connections and normalization

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Complete the newer CNN material without reprinting the full Topic 4 volume. Explain residual blocks and shape matching, count convolution-plus-BN parameters, and distinguish normalization statistics during training and inference.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Residual connections: learn a correction **3**

1.1 Basic blocks, bottlenecks and matching shapes 3

2 Batch normalization: computation and parameter count **4**

2.1 The slide's convolution-plus-BN count 4

2.2 Why it can help 4

3 Training, inference and normalization alternatives **4**

3.1 Answers to the slide's quick check 5

4 Updated slide-page guide **5**

This supplement covers the expanded material on PDF pages 51–62 of the current 111-page `4.1. CNNs.pdf`. It is also included in the updated full Topic 4 notes. If you already printed the earlier Topic 4 volume, print this supplement to add the new explanations.

1 Residual connections: learn a correction

Teacher slides: 4.1. CNNs, pp. 51–54, 60–62.

Increasing a plain network's depth can make optimization harder. If a deeper model has *higher training error*, this is not by itself the usual evidence for overfitting: overfitting concerns poor generalization despite fitting training data well.

A residual block learns $F(x)$ and adds a shortcut:

$$y = x + F(x).$$

If preserving the input is appropriate, the residual branch can learn $F(x) = 0$. Before any activation applied after the sum, the block then represents identity. This makes identity-like solutions easier to represent than requiring several ordinary layers to learn them indirectly.

For the residual sum, the Jacobian is

$$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}.$$

The identity term supplies a direct gradient contribution. It helps optimization but does not prove that gradients can never vanish, explode or cancel. Additional activations and projections must be included when differentiating a full block.

An in-principle argument is not a training guarantee

If the shapes and activation arrangement permit identity, a residual extension can represent the previous mapping by choosing a zero residual. This does not guarantee that training finds that solution or that the deeper model generalizes better.

1.1 Basic blocks, bottlenecks and matching shapes

A basic ResNet block uses two spatial convolutions. A bottleneck uses a 1×1 projection to reduce channels, a spatial convolution at that reduced width, then a 1×1 projection to expand again. The expensive spatial operation acts on fewer channels.

The slide compares two 3×3 convolutions at width 256 with a bottleneck $256 \to 64 \to 64 \to 256$, ignoring biases and normalization:

$$2(9)(256^2) = 1{,}179{,}648, \qquad 256(64) + 9(64^2) + 64(256) = 69{,}632.$$

The bottleneck uses about 17 times fewer convolution weights under these assumptions, not necessarily 17 times less execution time.

Residual addition requires matching spatial and channel dimensions. If a stage changes resolution or channels, a projection shortcut, often a strided 1×1 convolution, can match the shape. An identity shortcut has no learned parameters; a projection shortcut does.

Adding tensors preserves their common shape. Concatenating along channels adds their channel counts. Two $16 \times 16 \times 32$ feature maps give $16 \times 16 \times 32$ by addition, or $16 \times 16 \times 64$ by channel

concatenation. DenseNet and U-Net use concatenation in important parts of their designs; it is not interchangeable with residual addition.

2 Batch normalization: computation and parameter count

Teacher slides: 4.1. CNNs, pp. 55–56, 58.

Batch normalization (BN) controls activation scale using statistics inside the network, then applies a learned affine transformation. For values $a_1, \ldots, a_m$ in a normalization group,

$$\mu_B = \frac{1}{m}\sum_i a_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_i (a_i - \mu_B)^2, \qquad y_i = \gamma \frac{a_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

In a CNN, ordinary BN computes statistics separately for each channel over batch and spatial positions. For shape $N \times C \times H \times W$, one channel's group contains NHW values.

For C channels, C learned scales γ and C learned offsets β give $2C$ trainable parameters. Running means and variances are stored statistics, not coefficients learned by gradient descent. Implementations may also store bookkeeping such as a batch counter.

For values $(2, 4)$, the batch mean is 3 and population variance is 1. Ignoring ϵ for this arithmetic, normalization gives $(-1, 1)$. If $\gamma = 2, \beta = 0.5$, outputs are $(-1.5, 2.5)$. The learned affine step means final outputs do not have to retain mean zero and variance one.

2.1 The slide's convolution-plus-BN count

For a 3×3 convolution from 64 to 128 channels followed directly by BN, use no convolution bias:

$$N_{\text{conv}} = 9(64)(128) = 73{,}728, \qquad N_{\text{BN,trainable}} = 2(128) = 256.$$

Total trainable parameters are 73,984. Running statistics are not part of the trainable count.

A convolution bias immediately before BN is usually redundant: batch centering removes its constant channel shift and BN supplies a learned offset. If an exercise explicitly retains that bias, count the additional 128 stored trainable values rather than silently changing the architecture.

2.2 Why it can help

BN often makes optimization less sensitive to activation scale and can permit larger learning rates. Batch statistics also introduce noise with a mild regularizing effect. Initialization only sets the starting scale; normalization continues as weights change. The historical explanation in terms of internal covariate shift is not the only explanation of its benefits, as the slides themselves note.

3 Training, inference and normalization alternatives

Teacher slides: 4.1. CNNs, pp. 57–59, 62.

During ordinary BN training, normalization uses current mini-batch statistics and updates running estimates. During ordinary inference with tracked statistics, it uses stored estimates. Predictions then do not depend on which other samples share the evaluation batch, apart from other numerical or model effects.

For a convention in which m weights the new observation,

$$\mu_{\text{run,new}} = (1 - m)\mu_{\text{run,old}} + m\mu_B.$$

If the old mean is 2, the batch mean is 4 and $m = 0.1$, the new mean is 2.2. Some conventions name the complementary coefficient, so inspect the equation.

Evaluation mode and disabling gradient tracking are different controls. Turning off gradients alone does not switch BN to stored statistics or deactivate dropout. Freezing backbone weights also does not necessarily freeze BN running estimates if the backbone remains in training mode.

At inference, BN with fixed statistics is affine per channel and can often be folded into a preceding convolution. Small or unrepresentative training batches can give noisy statistics. The batch-size ranges on the slides are practical heuristics, not universal boundaries.

Method	Values sharing mean/variance	Course connection
Batch norm	Per channel, across batch and spatial positions in a CNN.	Training versus running-statistic inference.
Layer norm	Within a sample's chosen feature axes; usually each token's feature coordinates in a transformer.	Transformers and recurrent models.
Instance norm	Spatial positions within one sample and channel.	Image stylization.
Group norm	Spatial positions and a group of channels within one sample.	Small-batch CNN tasks.

The methods differ chiefly in which axes supply statistics. Transformer layer norm does not need a running batch mean and normally applies the same per-token computation during training and inference.

3.1 Answers to the slide's quick check

1. Higher training error in the deeper model points to optimization/representation issues; it is not alone evidence of overfitting.
2. A compatible residual block can represent identity through a zero residual, but successful training is not guaranteed.
3. The bias-free 3×3, $64 \to 128$ convolution plus BN has 73,984 trainable parameters.
4. Unexpected evaluation-batch dependence suggests checking training/evaluation mode and BN statistics; it is a diagnostic clue, not the only possible cause.
5. Layer norm uses within-sample feature statistics rather than BN's cross-sample per-channel statistics, and does not ordinarily rely on running batch estimates at inference.

4 Updated slide-page guide

Earlier printed notes referred to a 98-page version of the CNN deck. In the current 111-page deck, the main ranges are: operations and geometry (pp. 3–36), architecture designs including the

new material (pp. 37–66), transfer learning (pp. 67–74), training diagnostics (pp. 75–81), task extensions and engineering applications (pp. 82–105), the input-size exercise (p. 106), and Lab 3 reminders (pp. 108–110). The updated full Topic 4 PDF uses these current references.